Practical Session 6: Deep Dive into Classes and Objects

Student-Centered, Self-Paced Lab

🎯 Learning Outcomes Covered

By the end of this practical session, you will be able to:

1. **Analyze** a class design to identify encapsulation violations and propose fixes.
2. **Create** a well-encapsulated class with private fields, constructors, and public access methods.
3. **Evaluate** the necessity of getters/setters when fields could be made public.
4. **Design** a simple domain model (e.g., BankAccount, Student) with appropriate fields and methods.

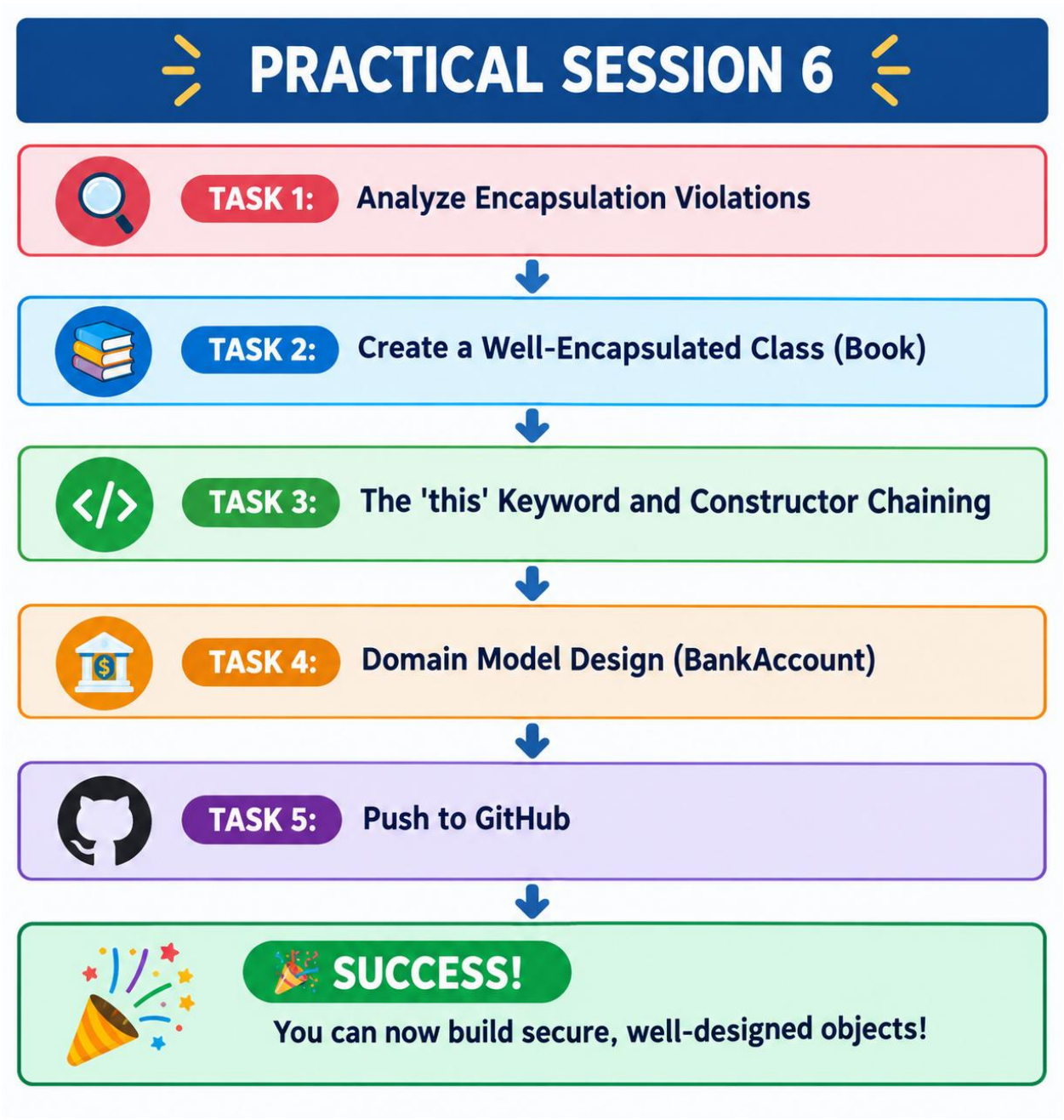
Time Required: 120–150 minutes
Instructor Intervention: None – you will figure it out yourself!

📋 Before You Begin

Ensure you have completed **Week 1 through Week 5 practical sessions**. You will need:

- VS Code open
- Your java-course folder with week5 subfolder
- Git configured and connected to GitHub

🗺️ **Your Roadmap**



TASK 1: Analyze Encapsulation Violations

Objective

Analyze two versions of a class to understand why encapsulation is critical for data security.

Step 1.1: Create a New Folder

1. Open **VS Code**.
2. Open your java-course folder.
3. Right-click the java-course folder → **New Folder**.
4. Name it: week6
5. Press Enter.

Step 1.2: Create the File

1. Right-click the week6 folder → **New File**.
2. Name the file: EncapsulationDemo.java
3. Press Enter.

Step 1.3: Write the Code

Copy and paste this code into EncapsulationDemo.java:

java

```
// This program demonstrates why encapsulation is important.
// It compares a poorly designed class (public fields) with a well-designed class (private fields).


// ============================================================
// POORLY DESIGNED CLASS (Violates Encapsulation)
// ============================================================
class BadStudent {
    // These fields are PUBLIC, meaning ANY other class can change them directly.
    // There is no validation. A student's age could be set to -100!
    public String name;
```

```
    public int age;
    public double gpa;
}

// ============================================================
// WELL-DESIGNED CLASS (Follows Encapsulation)
// ============================================================
class GoodStudent {
    // These fields are PRIVATE, meaning they are hidden from other classes.
    // They can only be accessed or modified through the public methods below.
    private String name;
    private int age;
    private double gpa;

    // Constructor to initialize the fields safely.
    public GoodStudent(String name, int age, double gpa) {
        // We use 'this' to refer to the object's fields, and validate the data.
        this.name = name;
        setAge(age);   // Use setter for validation
        setGpa(gpa);   // Use setter for validation
    }

    // GETTER for name
    public String getName() {
        return name;
    }
```

```
// SETTER for name
public void setName(String name) {
    this.name = name;
}

// GETTER for age
public int getAge() {
    return age;
}

// SETTER for age with VALIDATION
public void setAge(int age) {
    // Only allow ages between 0 and 150.
    if (age >= 0 && age <= 150) {
        this.age = age;
    } else {
        System.out.println("Error: Invalid age. Age must be between 0 and 150.");
    }
}

// GETTER for GPA
public double getGpa() {
    return gpa;
}
```

```
// SETTER for GPA with VALIDATION
    public void setGpa(double gpa) {
        // Only allow GPAs between 0.0 and 4.0.
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        } else {
            System.out.println("Error: Invalid GPA. GPA must be between 0.0 and 4.0.");
        }
    }
}

// ============================================================
// MAIN CLASS
// ============================================================
public class EncapsulationDemo {
    public static void main(String[] args) {

        System.out.println("=== TESTING BAD STUDENT ===");

        // Create a BadStudent object.
        BadStudent bad = new BadStudent();

        // Directly accessing and modifying public fields. No validation!
        bad.name = "Alice";
        bad.age = -100;     // No error! This is impossible in real life.
        bad.gpa = 15.5;     // No error! GPA cannot be 15.5.
```

```
        // Print the corrupted data.
        System.out.println("BadStudent Name: " + bad.name);
        System.out.println("BadStudent Age: " + bad.age + " (Corrupted!)");
        System.out.println("BadStudent GPA: " + bad.gpa + " (Corrupted!)");

        System.out.println("\n=== TESTING GOOD STUDENT ===");

        // Create a GoodStudent object using the constructor.
        GoodStudent good = new GoodStudent("Bob", 20, 3.5);

        // Print valid data using getters.
        System.out.println("GoodStudent Name: " + good.getName());
        System.out.println("GoodStudent Age: " + good.getAge());
        System.out.println("GoodStudent GPA: " + good.getGpa());

        // Attempt to corrupt the data using setters.
        System.out.println("\nAttempting to set invalid age and GPA...");
        good.setAge(-100); // Should trigger validation error
        good.setGpa(15.5); // Should trigger validation error

        // Data remains safe and unchanged.
        System.out.println("GoodStudent Age after invalid attempt: " + good.getAge());
        System.out.println("GoodStudent GPA after invalid attempt: " + good.getGpa());
    }
}
```

Step 1.4: Run the Program

Save and run. Observe how BadStudent allows impossible data, while GoodStudent blocks it.

Expected Output:

text

=== TESTING BAD STUDENT ===

BadStudent Name: Alice

BadStudent Age: -100 (Corrupted!)

BadStudent GPA: 15.5 (Corrupted!)

=== TESTING GOOD STUDENT ===

GoodStudent Name: Bob

GoodStudent Age: 20

GoodStudent GPA: 3.5

Attempting to set invalid age and GPA...

Error: Invalid age. Age must be between 0 and 150.

Error: Invalid GPA. GPA must be between 0.0 and 4.0.

GoodStudent Age after invalid attempt: 20

GoodStudent GPA after invalid attempt: 3.5

Step 1.5: Answer These Questions

1. Why is BadStudent considered a bad design?
2. How does GoodStudent prevent invalid data?
3. What is the purpose of private?
4. What is the purpose of getters and setters?

✅ **Task 1 Complete**

- □ I created EncapsulationDemo.java

- □ I ran the program successfully
- □ I understand the difference between public and private fields

---

TASK 2: Create a Well-Encapsulated Class (Book)

Objective

Create a Book class from scratch that strictly follows encapsulation principles.

Step 2.1: Create the File

Create a new file called Book.java in the week6 folder.

Step 2.2: Write the Code

java

```
// This class represents a Book and follows encapsulation principles.


public class Book {


    // ============================================================
    // 1. PRIVATE FIELDS (Attributes)
    // ============================================================
    // These are hidden from the outside world.
    private String title;
    private String author;
    private double price;
    private boolean isAvailable;
```

```
// ============================================================
 // 2. CONSTRUCTOR
 // ============================================================
 // Initializes the object when it is created.
 public Book(String title, String author, double price) {
    this.title = title;
    this.author = author;
    setPrice(price); // Use setter to validate
    this.isAvailable = true; // New books are available by default
 }

 // ============================================================
 // 3. GETTERS (Accessors)
 // ============================================================
 // Used to READ private fields safely.
 public String getTitle() {
    return title;
 }

 public String getAuthor() {
    return author;
 }

 public double getPrice() {
    return price;
 }
```

```
public boolean isAvailable() {
    return isAvailable;
}

// ============================================================
// 4. SETTERS (Mutators)
// ============================================================
// Used to MODIFY private fields safely with validation.
public void setTitle(String title) {
    this.title = title;
}

public void setAuthor(String author) {
    this.author = author;
}

public void setPrice(double price) {
    // Business Rule: Price cannot be negative
    if (price >= 0) {
        this.price = price;
    } else {
        System.out.println("Error: Price cannot be negative.");
    }
}
```

```
    // ============================================================
    // 5. BEHAVIORAL METHODS
    // ============================================================
    // Actions the book can perform.
    public void borrowBook() {
        if (isAvailable) {
            isAvailable = false;
            System.out.println("You have borrowed: " + title);
        } else {
            System.out.println("Sorry, " + title + " is currently out of stock.");
        }
    }

    public void returnBook() {
        isAvailable = true;
        System.out.println("You have returned: " + title);
    }

    // Method to display all details
    public void displayBookInfo() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.printf("Price: KSh %.2f%n", price);
        System.out.println("Status: " + (isAvailable ? "Available" : "Borrowed"));
        System.out.println("------------------------");
    }
```

}

Step 2.3: Create a Test File

Create a new file called BookTest.java to test your new class.

java

```
public class BookTest {
    public static void main(String[] args) {
        // Create a new Book object
        Book myBook = new Book("Java Programming", "John Doe", 1500.00);

        // Display initial info
        myBook.displayBookInfo();

        // Test valid and invalid setters
        System.out.println("Attempting to change price to -500...");
        myBook.setPrice(-500); // Should trigger error

        System.out.println("\nBorrowing the book...");
        myBook.borrowBook();
        myBook.displayBookInfo();

        System.out.println("Returning the book...");
        myBook.returnBook();
        myBook.displayBookInfo();
    }
}
```

Step 2.4: Run the Program

Save both files and run BookTest.java. Ensure the validation works.

✅ **Task 2 Complete**

- □ I created Book.java with proper encapsulation
- □ I created BookTest.java to test it
- □ I ran the program successfully

TASK 3: The 'this' Keyword and Constructor Chaining

Objective

Use the this keyword to handle name collisions and chain constructors to avoid code duplication.

Step 3.1: Create the File

Create a new file called Employee.java in the week6 folder.

Step 3.2: Write the Code

java

```
// This class demonstrates constructor chaining and the 'this' keyword.


public class Employee {


    // Private fields
    private int id;
    private String name;
    private double salary;
    private String department;
```

```
// ============================================================
    // CONSTRUCTOR 1: Master Constructor (Takes all arguments)
    // ============================================================
    public Employee(int id, String name, double salary, String department) {
        // 'this' distinguishes the instance variable from the parameter.
        // Without 'this', id = id would assign the parameter to itself!
        this.id = id;
        this.name = name;
        this.salary = salary;
        this.department = department;
    }

    // ============================================================
    // CONSTRUCTOR 2: Overloaded (No department specified)
    // ============================================================
    public Employee(int id, String name, double salary) {
        // CONSTRUCTOR CHAINING: this() calls another constructor.
        // It MUST be the first line in the constructor.
        // Here, we call Constructor 1 and provide a default department.
        this(id, name, salary, "Unassigned");
    }

    // ============================================================
    // CONSTRUCTOR 3: Overloaded (Only id and name)
    // ============================================================
    public Employee(int id, String name) {
```

```
        // Chain to Constructor 2, providing a default salary.
        // Constructor 2 will then chain to Constructor 1.
        this(id, name, 25000.0);
    }

    // ============================================================
    // METHODS
    // ============================================================
    public void giveRaise(double amount) {
        if (amount > 0) {
            // 'this' is optional here, but clarifies that we are modifying the instance field.
            this.salary += amount;
            System.out.println(name + " received a raise. New salary: " + this.salary);
        }
    }

    public void displayInfo() {
        System.out.println("ID: " + id + " | Name: " + name +
                           " | Salary: " + salary + " | Dept: " + department);
    }
}
```

Step 3.3: Create a Test File

Create a new file called EmployeeTest.java:

java

```
public class EmployeeTest {
    public static void main(String[] args) {
```

```
        System.out.println("=== TESTING CONSTRUCTORS ===");

        // Using Constructor 1 (All parameters)
        Employee e1 = new Employee(101, "Alice", 50000.0, "IT");
        e1.displayInfo();

        // Using Constructor 2 (No department)
        Employee e2 = new Employee(102, "Bob", 45000.0);
        e2.displayInfo();

        // Using Constructor 3 (Only ID and Name)
        Employee e3 = new Employee(103, "Charlie");
        e3.displayInfo();

        System.out.println("\n=== TESTING METHODS ===");
        e3.giveRaise(5000.0);
        e3.displayInfo();
    }
}
```

Step 3.4: Run the Program

Save and run EmployeeTest.java. Notice how the constructors reuse code via this().

✅ **Task 3 Complete**

- □ I created Employee.java
- □ I created EmployeeTest.java
- □ I understand how this() chains constructors

TASK 4: Domain Model Design (BankAccount)

Objective

Design a realistic BankAccount class that uses encapsulation to enforce banking business rules.

Step 4.1: Create the File

Create a new file called BankAccount.java in the week6 folder.

Step 4.2: Write the Code

java

```
// This class represents a Bank Account and enforces strict business rules using encapsulation.


public class BankAccount {


    // Private fields
    private String accountNumber;
    private String accountHolderName;
    private double balance;


    // Constructor
    public BankAccount(String accountNumber, String accountHolderName, double
initialBalance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;


        // Validate initial balance
        if (initialBalance >= 0) {
            this.balance = initialBalance;
        } else {
```

```
        this.balance = 0;
        System.out.println("Warning: Initial balance cannot be negative. Set to 0.");
    }
}

// ============================================================
// GETTERS (Read-only for account number and holder)
// ============================================================
public String getAccountNumber() {
    return accountNumber;
}

public String getAccountHolderName() {
    return accountHolderName;
}

public double getBalance() {
    return balance;
}

// Note: There is NO setBalance() method!
// The balance can ONLY be changed via deposit() and withdraw().
// This prevents external classes from arbitrarily changing the balance.

// ============================================================
// BUSINESS LOGIC METHODS
```

```
// ============================================================

// Method to deposit money
public void deposit(double amount) {
    // Validation: Amount must be positive
    if (amount > 0) {
        balance += amount;
        System.out.printf("Deposited: KSh %.2f. New Balance: KSh %.2f%n", amount, balance);
    } else {
        System.out.println("Error: Deposit amount must be positive.");
    }
}

// Method to withdraw money
public void withdraw(double amount) {
    // Validation 1: Amount must be positive
    if (amount <= 0) {
        System.out.println("Error: Withdrawal amount must be positive.");
        return; // Exit the method early
    }

    // Validation 2: Sufficient funds check
    if (amount > balance) {
        System.out.println("Error: Insufficient funds. Available balance: KSh " + balance);
        return;
    }
```

```
        // If all validations pass, process the withdrawal
        balance -= amount;
        System.out.printf("Withdrew: KSh %.2f. New Balance: KSh %.2f%n", amount, balance);
    }
}
```

Step 4.3: Create a Test File

Create a new file called BankTest.java:

```
java
public class BankTest {
    public static void main(String[] args) {
        System.out.println("=== OPENING ACCOUNT ===");
        // Create a new account with an initial balance of 5000
        BankAccount myAccount = new BankAccount("ACC-001", "Alice Wanjiku", 5000.00);
        System.out.println("Account Holder: " + myAccount.getAccountHolderName());
        System.out.println("Initial Balance: KSh " + myAccount.getBalance());

        System.out.println("\n=== DEPOSITING ===");
        myAccount.deposit(2500.50);
        myAccount.deposit(-100); // Should fail (negative amount)

        System.out.println("\n=== WITHDRAWING ===");
        myAccount.withdraw(2000);
        myAccount.withdraw(10000); // Should fail (insufficient funds)
        myAccount.withdraw(-50);   // Should fail (negative amount)
```

```
        System.out.println("\n=== FINAL STATE ===");
        System.out.println("Final Balance: KSh " + myAccount.getBalance());
    }
}
```

Step 4.4: Run the Program

Save and run BankTest.java. Notice how the object protects itself from being put into an invalid state (like a negative balance).

✅ **Task 4 Complete**

- □ I created BankAccount.java
- □ I created BankTest.java
- □ I tested all the business rules (deposit, withdraw, insufficient funds)

TASK 5: Push to GitHub

Step 5.1: Stage and Commit

1. Open the **Source Control** panel in VS Code.
2. Stage all new files in week6.
3. Commit with message: Week 6: Classes & Objects - Encapsulation, Constructors, Getters/Setters
4. Push to GitHub.

Step 5.2: Verify on GitHub

Go to your repository and confirm all Week 6 files are there.

✅ **Task 5 Complete**

- □ I committed and pushed all files
- □ I verified on GitHub

🏆 Final Self-Assessment Checklist

- □ I can explain why public fields are dangerous (Encapsulation violation).
- □ I can create private fields and provide public getters/setters.
- □ I can use validation in setters to prevent bad data.
- □ I understand how to use the this keyword.
- □ I can use this() to chain constructors.
- □ I can design a domain model (like BankAccount) with strict business logic.
- □ I pushed all Week 6 code to GitHub.

✓ Troubleshooting Guide

Problem	Solution
cannot find symbol: class Book	Ensure both files are in the same folder. Ensure the file name matches the class name.
Getters/Setters not working	Check that the method names match exactly (case-sensitive).
NullPointerException	You might be trying to call a method on an object that hasn't been initialized with new.
Logical error (e.g., balance wrong)	Trace your math. Are you using + instead of -? Are your validation conditions correct?

✓ Reflection Questions

1. Why do we make fields private instead of public?
2. What is the main purpose of a constructor?
3. How does the this keyword help avoid confusion?
4. In the BankAccount class, why didn't we create a setBalance() method?

🎓 What You've Achieved

You have just:

- Mastered **Encapsulation** (hiding data, exposing safe methods).
- Learned how to use **Constructors** to initialize objects.
- Used **Constructor Chaining** to write cleaner code.
- Built a **secure domain model** (BankAccount) that prevents invalid data.

You are now writing professional, secure object-oriented code!

✓ Next Steps

In Week 7, you will:

1. Explore **Inheritance** (parent-child relationships).
2. Learn how to use the extends keyword.
3. Discover how to **Override** methods.
4. Continue pushing your code to GitHub.

END OF PRACTICAL SESSION 6

This response is AI-generated, for reference only.